Home SOC Lab Write-up

Akinloluwa Euba · October 2026

Summary

I built a virtual enterprise network on one laptop and used it to run the full defender loop: build, monitor, attack, detect, triage. All 5 simulated attacks on a domain-joined Windows 11 client were detected by the Wazuh SIEM (Security Information and Event Management) and mapped to MITRE ATT&CK techniques.

- **Environment:** VMware Workstation on a 32 GB / 500 GB SSD laptop, running an OPNsense firewall, a Windows Server 2025 domain controller (`lab.local`), a Windows 11 client and an Ubuntu Server running Wazuh 4.14.8.
- **Telemetry:** Wazuh agents plus Sysmon (SwiftOnSecurity config) on both Windows hosts, centrally configured from the Wazuh manager.
- **Result:** 5 of 5 attack techniques detected; one level 15 alert triaged and closed as a false positive; one detection limitation documented with a tuning recommendation.
- **Build effort:** 6–15 October 2026, including diagnosing and fixing a corrupted firewall system file, a half-allocated server disk and a broken domain trust.

Lab architecture

Four virtual machines run on one laptop in VMware Workstation, split across two lab networks joined by an OPNsense firewall.

Machine	Role	OS	IP address
OPNsense-Router	Firewall, router, NAT to the internet	OPNsense	WAN 192.168.110.128 (from VMware NAT); LAN 192.168.10.1 and 192.168.20.1
DC01	Domain controller and DNS for `lab.local`	Windows Server 2025	192.168.10.10
Ubuntu-Server	Wazuh SIEM; dashboard at https://192.168.10.20	Ubuntu Server	192.168.10.20
Win11-Client	Domain-joined client; Wazuh agent `WN11`	Windows 11 Pro	192.168.20.50

Log flow: DC01 and Win11 each run a Wazuh agent and Sysmon. The agents send Windows and Sysmon events to the Wazuh manager on Ubuntu, which stores them in the indexer. Alerts are reviewed in the Wazuh dashboard from Win11's browser.

Build overview

The lab was built in six stages, each ending in a VM snapshot so a later failure could not undo it.

1. **Hypervisor and virtual networks:** VMware Workstation with a NAT network for internet access and separate host-only segments for the lab LANs.
2. **Firewall / router:** OPNsense routes between the 192.168.10.0/24 and 192.168.20.0/24 segments and NATs lab traffic to the internet through its WAN interface.

3. **Active Directory:** Windows Server 2025 promoted to domain controller for `lab.local`, static IP 192.168.10.10, DNS forwarding to 8.8.8.8 for internet names. Windows 11 joined to the domain and logged in with domain accounts.
4. **SIEM server:** Ubuntu Server with a permanent netplan static IP (192.168.10.20). Wazuh all-in-one (indexer, manager, dashboard) installed from pre-downloaded packages to survive a slow link.
5. **Endpoint telemetry:** Wazuh agents deployed to DC01 and Windows 11. Sysmon installed on both with the SwiftOnSecurity config. Sysmon's event channel added once in the manager's shared `agent.conf`, which pushes it to every agent.
6. **Attack simulation:** five techniques run on Windows 11, each checked in Wazuh Threat Hunting and documented with screenshots.

Tools: VMware Workstation, OPNsense, Windows Server 2025, Windows 11 Pro, Ubuntu Server, Wazuh 4.14.8, Sysmon, PowerShell, netplan, LVM.

Detection testing

All 5 techniques were detected. Tests ran on 13 October 2026 on the Windows 11 client (agent `WN11`, 192.168.20.50) as `LAB\Administrator`.

#	Technique (MITRE)	Command run	Wazuh alert (rule ID, level)	Result
1	Brute Force (T1110)	`runas /user:lab\fakeuser cmd`, wrong password ×5	Logon Failure: unknown user or bad password (60122, L5), 5 alerts	Detected
2	Create Account: Local (T1136.001)	`net user hacker <password> /add`	User account enabled or created (60109, L8); User account changed (60110, L8)	Detected
3	Account Manipulation (T1098)	`net localgroup administrators hacker /add`	Administrators Group Changed (60154, L12)	Detected
4	Clear Windows Event Logs (T1070.001)	`wevtutil cl Security`	The audit log was cleared (63103, L5)	Detected
5	PowerShell (T1059.001)	`powershell -enc VwByAGkA…` (encoded `Write-Output hello`)	PowerShell process spawned PowerShell instance (92027, L4), full command line captured	Detected (see Findings)

Test 4 also shows why central logging matters: the Security log on the client was emptied, but Wazuh already held copies of every earlier event. The `hacker` test account was deleted after testing.

Findings

1. Encoded PowerShell was caught by behaviour, not by the encoding

Rule 92027 fired because one PowerShell process started another, a parent-child pattern common in attacks. It did not fire because of `-enc`. Sysmon captured the full command line, so an analyst could decode the payload. An attacker launching encoded PowerShell from another parent, such as a document macro, may not trip this rule.

Recommendation: add a custom rule that alerts on PowerShell command lines containing `-enc` or `-EncodedCommand`.

2. Level 15 alert triaged as a false positive

Rule 92213, "Executable file dropped in folder commonly used by malware", fired 3 times at level 15, the highest severity used by built-in rules.

- **Process:** `C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe`
- **File:** `C:\Users\Administrator\AppData\Local\Temp\__PSScriptPolicyTest_akjxwsjp.dfy.ps1`
- **User:** `LAB\Administrator`

PowerShell writes a throwaway `__PSScriptPolicyTest_*` script to Temp at every launch to check script policy. Each 92213 alert (13:29:43, 13:42:42, 13:45:38) fired in the same second as a 92027 PowerShell launch from Test 5. Verdict: benign, caused by the test itself.

Recommendation: exclude `__PSScriptPolicyTest_` file names from rule 92213. Left untuned, every PowerShell launch raises a top-severity alert, which trains analysts to ignore it.

3. Extra detections from the account tests

The `net user` and `net localgroup` commands also triggered "A net.exe account discovery command was initiated" (92039) and "Discovery activity spawned via PowerShell execution" (92033). That is Account Discovery (T1087), caught without a dedicated test. Wazuh also logged "System time changed" (60132) when I corrected the client's clock, an example of benign admin activity in the SIEM.

4. Observed, not investigated

- "Application Compatibility Database launched" (92058, L12) at 13:47:28.
- "Executable dropped in Windows root folder" (92217, L6) at 13:56:36 and 14:03:26.

Both appeared after testing ended and are likely Windows Update or Defender activity. They were not triaged.

Troubleshooting log

Newest first. Each row is a fault actually hit during the build, with the root cause found and the fix applied.

Date	Symptom	Root cause	Fix	Result
13 Oct	`dcdiag` reported trust-account failures from `DESKTOP-MCETGN0`; `Test-ComputerSecureChannel` returned False	Win11's computer account was missing from Active Directory, so the secure channel had nothing to repair against	`New-ADComputer -Name DESKTOP-MCETGN0` on DC01; then `Test-ComputerSecureChannel -Repair -Credential lab\Administrator` on Win11	Secure channel repaired (`True`)
13 Oct	Win11 clock days behind; SIEM alert timestamps wrong	Likely snapshot restores and host crashes	Corrected Win11's date/time manually (Settings → Time & language)	Clock correct; alerts timestamped properly
11 Oct	Win11 could not resolve internet names; `Resolve-DnsName packages.wazuh.com` timed out	Win11's preferred DNS still pointed at DC01's **old** IP (.152)	Set Win11's preferred DNS to 192.168.10.10	Lookups resolved; agent install succeeded
9 Oct	Wazuh indexer stuck "activating" for 20+ minutes; dashboard timed out	Wazuh's vulnerability database (~17 GB: 9.1 GB in `/var/ossec/tmp`, 7.8 GB in `queue/vd`) filled the 28 GB disk; indexer could not write and never reported ready	Disabled vulnerability detection in `ossec.conf` (`<enabled>no</enabled>`); deleted contents of `/var/ossec/tmp`, `/var/ossec/queue/vd`, `/var/ossec/queue/vd_updater`; restarted indexer, manager, dashboard in order	Indexer started; dashboard reachable; ~16 GB free

Date	Symptom	Root cause	Fix	Result
8–9 Oct	Wazuh installer failed a third time: "ports 1515 and 55000 already in use; manager package could not be removed"	Failed rollback from a previous run left the Wazuh manager half-removed but still running; broken `prerm`/`postrm` scripts blocked purge	Removed broken `/var/lib/dpkg/info/wazuh-*rm` and `filebeat*rm` scripts; `dpkg --purge --force-all` all four packages; deleted leftover folders; clean reinstall	Wazuh installed successfully end to end
8 Oct	Wazuh installer failed: "No space left on device" during dashboard install	Ubuntu's LVM volume was only half allocated: 14 GB used of a 28 GB virtual disk; the other 14 GB sat unallocated (VFree)	`lvextend -l +100%FREE /dev/ubuntu-vg/ubuntu-lv` + `resize2fs`; `apt update` to repair package lists	Root volume grew to 28 GB with 19 GB free; the next install attempt was then blocked by leftovers (see 8–9 Oct row)
7 Oct	Wazuh indexer package download crawled at ~470 kbps; live installer kept failing on slow Wi-Fi	Slow link; live installer cannot resume interrupted downloads	Pre-downloaded packages with `wget -c` (resumable) into `/var/cache/apt/archives/`; re-ran installer	Installer found packages locally and skipped downloads
7 Oct	OPNsense boot took ~60 minutes; no internet; DNS SERVFAIL; Wazuh installer failed on GPG key	Host crashes had **bloated** `/etc/group` to 41 KB of scrambled records (normal ~0.5 KB); `configd` crashed on `getgrgid(): gid not found: 0`; every boot task timed out (~20 s each, dozens stacked)	Restored `/var/backups/group.bak` → `/etc/group`; restarted `configd`; `configctl interface reconfigure wan` returned OK	Boot dropped to under 2 minutes; gateway applied automatically; internet restored
7 Oct	OPNsense WAN had a valid IP but no default route ("no route to host")	WAN DHCP lease arrived without a gateway; `configctl` was broken by the bloated group file	Added manual default route `192.168.110.2` as a stopgap; permanent fix was the group-file repair above	Gateway persisted across reboot after repair
7 Oct	Ubuntu had two IPs (192.168.10.125 DHCP + 192.168.10.20 manual); netplan errors	Conflicting netplan files: `00-installer-config.yaml` had `8.8.8.8` misplaced under `addresses:`; `50-cloud-init.yaml` had `[8.8.8.8] [1.1.1.1]` instead of comma-separated	Rewrote `00-installer-config.yaml` with correct indentation; moved `50-cloud-init.yaml` aside; `chmod 600`; disabled cloud-init network config	Ubuntu came up with only 192.168.10.20; internet and DNS working
7 Oct	DC01's static IP (192.168.10.10) not in effect; DC01 running on DHCP .152	Static IP had not been applied; DC01 was on DHCP .152	Set DC01 to .10 statically	DC01 reachable on .10
6 Oct	OPNsense WAN lost its lease after laptop sleep/resume; NAT broken	VMware NAT and DHCP services lost track of the host's Wi-Fi after sleep	Restarted `VMware NAT Service` and `VMware DHCP Service`; power-cycled OPNsense; `configctl interface reconfigure wan`	WAN lease renewed; internet restored
6 Oct	`ping 8.8.8.8` from Ubuntu returned "Destination Host Unreachable" from 192.168.10.1	OPNsense had no path to the internet (WAN side), not a DNS problem	Diagnosed from OPNsense shell with `ping` and `ifconfig em0`; fixed NAT link as above	Ping succeeded

Known limitations and next steps

Limitations:

- **Vulnerability detection is disabled** on the Wazuh manager until the Ubuntu VM gets more disk. The feature's database (~17 GB) filled a 28 GB disk. Log collection, alerting, agents and Sysmon are unaffected.
- **Host stability.** The host laptop crashed several times during the build, which corrupted a firewall system file and left VMware lock files behind. Mitigated with snapshots at each milestone, power settings (sleep disabled) and keeping the laptop on mains power.
- **DC01 still reports two non-critical `dcdiag` errors** related to Secure Boot, which is switched off on the VM. Domain logins and joins work normally.
- **Encoded PowerShell detection is partial.** Test 5 was caught by a Sysmon-based rule that fires when PowerShell spawns PowerShell — not because of the `-enc` flag itself. A direct `-EncodedCommand` invocation from a non-PowerShell parent may not trigger.
- **One level-15 alert class is untuned.** "Executable file dropped in folder commonly used by malware" fires on PowerShell's own `__PSScriptPolicyTest_*.ps1` in `%TEMP%`, producing top-severity false positives. Triaged and documented; rule not yet tuned.
- **The lab runs on a single host** with no redundancy; a host crash interrupts everything, which is why snapshots at each milestone were essential.

Next steps:

1. **Enlarge the Ubuntu VM disk** (to ~60 GB) and re-enable Wazuh vulnerability detection.
2. **Write a custom Wazuh rule** to flag PowerShell launched with `-enc` / `-EncodedCommand`, then re-run Test 5 to confirm it fires — closing the documented detection gap.
3. **Tune the level-15 Sysmon rule** to ignore `__PSScriptPolicyTest_*.ps1`, reducing false positives.
4. **Harden the host** against crashes (check Hyper-V, Core Isolation and Virtualization-Based Security settings; confirm sleep is disabled).
5. **Add a second Windows client** (W10) to the domain to broaden detection coverage.
6. **Re-run `dcdiag /q`** after the IP and DNS cleanups to confirm the remaining Secure Boot errors are the only ones.
7. **Expand the attack simulation** beyond the five tests (e.g. Atomic Red Team) and map each to MITRE ATT&CK.
8. **Forward OPNsense firewall logs to Wazuh** so network-level events sit alongside endpoint alerts.

Skills demonstrated

- **Network segmentation and routing** — built a two-subnet lab behind OPNsense with NAT to the internet; configured WAN via VMware NAT and LAN gateways on 192.168.10.1 and 192.168.20.1.
- **Firewall administration** — installed and configured OPNsense; diagnosed a missing default gateway; used `configctl interface reconfigure wan` and manual route addition to restore connectivity.
- **Linux system administration** — configured static IPs via netplan (correct indentation, cloud-init disabled), extended an LVM volume live with `lvextend` + `resize2fs`, managed services with `systemctl`, and diagnosed disk exhaustion with `df`, `du`, `vgs`.
- **FreeBSD/OPNsense recovery** — traced a 60-minute boot to a bloated `/etc/group` (41 KB of scrambled records), used `pkg check -s -a` to rule out package damage, restored from `/var/backups/group.bak`, and verified with a timed reboot.

- **Windows Server / Active Directory** — promoted DC01 to a domain controller for `lab.local`, joined Win11 to the domain, created domain users, set static IP and DNS, and repaired a broken computer-account trust with `Test-ComputerSecureChannel -Repair`.
- **DNS troubleshooting** — diagnosed Win11 name-resolution failures to a stale preferred-DNS address (.152 instead of .10) and corrected it.
- **Time synchronisation** — identified and corrected client clock drift; understood its impact on SIEM timestamps and Kerberos.
- **SIEM deployment and operation** — installed Wazuh 4.14.8 (indexer, manager, dashboard) all-in-one on Ubuntu; deployed Wazuh agents to DC01 and Win11; confirmed both Active in the dashboard.
- **Endpoint telemetry** — installed Sysmon with the SwiftOnSecurity config on both Windows machines; verified service status and event flow into Wazuh.
- **Log analysis and alert triage** — investigated alerts in Wazuh Threat Hunting; expanded events to read `data.win.eventdata.targetFilename` and `eventdata.image`; triaged a level-15 false positive (PowerShell's `__PSScriptPolicyTest_*.ps1`) and documented the tuning recommendation.
- **Attack simulation and detection validation** — ran five safe tests mapped to MITRE ATT&CK: brute-force logon (T1110), account creation (T1136), admin group modification (T1098), audit log clearing (T1070.001), and encoded PowerShell (T1059.001); confirmed detection for all five and documented the nuance in Test 5.
- **Detection gap analysis** — identified that encoded PowerShell was caught via parent→child process behaviour rather than the encoding itself, and recommended a custom rule.
- **Virtualisation** — built and ran four VMs on VMware Workstation; diagnosed and fixed NAT service failures after host sleep; recovered from `.lck` lock errors after host crashes; used snapshots as recovery points at each milestone.
- **Troubleshooting methodology** — followed symptoms upstream (symptom → logs → root cause → fix → verification) across networking, Linux, Windows and the SIEM; documented every fix in a structured troubleshooting log.
- **Technical documentation** — produced this write-up, including the architecture and IP table, detection results, findings, limitations and skills.